
SMART INDIA HACKATHON 2026 · PROBLEM STATEMENT 26155 · NTRO

NIRIKSHAK

Self-learning, vendor-agnostic network security compliance auditor

AI SUGGESTS · RULES DECIDE

Team Atlantis

Project Report — complete system description

8 September 2026

— Contents

- 1 The problem, and the idea
- 2 What the system does — the pipeline
- 3 The six non-negotiable rules
- 4 Innovation I — vendor packs as data
- 5 Innovation II — the administrator confirmation loop
- 6 Innovation III — abstention as a first-class result
- 7 Innovation IV — the vetted remediation library
- 8 Innovation V — the tamper-evident audit chain
- 9 Where AI is used, why, and how it is contained
- 10 The technology stack, and why each choice
- 11 The interface, screen by screen
- 12 Security posture
- 13 Measured results
- 14 What is built — feature inventory
- 15 What is deliberately not claimed
- 16 Known defects and open work
- 17 Engineering discipline
- 18 Summary

1 The problem, and the idea

1.1 The problem

A large network runs devices from many vendors — Cisco, Juniper, Arista, Palo Alto and others — and each expresses the same security control in different syntax. "Is Telnet disabled?" is `transport input ssh` on Cisco IOS, `delete system services telnet` on Junos, and `management telnet` followed by `shutdown` on Arista EOS. Auditing a fleet for compliance therefore means either employing someone who knows every platform, or accepting that most devices go unchecked.

Existing tools tend to fail in one of two ways. Either they support a fixed list of vendors and are useless the moment a device outside it appears, or they use a language model to interpret configurations and produce confident answers nobody can verify. In a security audit the second failure is worse than the first: a wrong PASS is a vulnerability that has been formally signed off.

1.2 The idea

NIRIKSHAK reads configuration files that were exported offline, translates vendor-specific syntax into one vendor-neutral security schema, evaluates that schema with a deterministic rule engine, and produces findings that each cite the exact configuration line they rest on.

When it meets a line no vendor pack recognises, **it does not guess**. It clusters the unknown line, ranks candidate meanings, and asks an administrator. Their confirmed answer is compiled into a versioned vendor pack, and the next parse recognises that syntax deterministically — permanently, and without a code release.

THE CENTRAL DESIGN COMMITMENT

AI suggests. Rules decide. No model ever issues a compliance verdict. A machine-learning suggestion reaches a verdict by exactly one route: a human confirms it, it becomes pack *data*, and the deterministic parser matches it on the next run. This is enforced by the import graph and by a database constraint, not by intention.

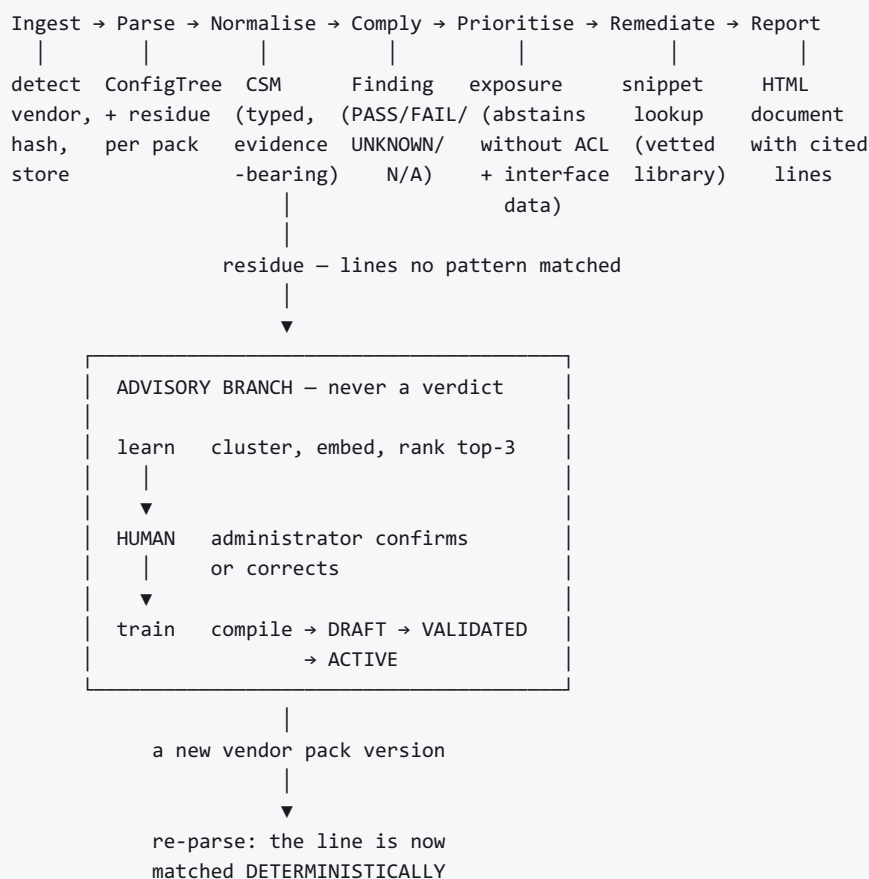
1.3 Scale of the implementation

109 PYTHON MODULES	16.8k BACKEND LINES	8.2k FRONTEND LINES	1,887 BACKEND TESTS	34 UI TESTS
15 API PACKAGES	28 HTTP ENDPOINTS	23 DECISION RECORDS	60 NUMBERED DECISIONS	19.4k TEST LINES

Test code exceeds application code by roughly 15%. That ratio is deliberate: in a system whose product is trustworthy verdicts, the tests are not overhead, they are the evidence.

2 What the system does — the pipeline

Seven deterministic stages produce every verdict. The advisory branch sits *beside* this spine and can never enter it.



The deterministic spine, and the advisory loop that feeds it data but never verdicts.

2.1 One audit, end to end

A single request — `POST /compliance/audits?file_id=...` — traverses the whole spine. Each arrow is a typed contract.

#	STAGE	WHAT HAPPENS
1	Read	The stored blob is fetched by its content hash.
2	Parse	A <code>ConfigTree</code> is built, the pack's patterns applied, and a <code>ParseResult</code> returned carrying fields, the pack version that read them, and the residue — every node no pattern matched.
3	Identity	Hostname, model, OS version and serial are read using the pack's identity patterns.
4	Normalise	Resolves what every <i>absent</i> field means: a documented platform default, a control the platform cannot express, or an honest UNKNOWN.
5	Evaluate	One <code>Finding</code> per applicable rule — PASS, FAIL, UNKNOWN or NOT_APPLICABLE — each carrying the evidence it rests on or the reason it abstained.
6	Analyse	ACL interval logic runs on a separate rail from findings.
7	Prioritise	Exposure is assessed per finding; only what could be determined is ranked.
8	Persist	The run and its findings are written to the operational store.
9	Queue	Residue becomes the durable training queue, replacing that file's previous entries.
10	Attest	An <code>AUDIT_RUN</code> record is appended to the hash chain — counts, identifiers and versions only.

2.2 The canonical security model

All vendor syntax maps to one vendor-neutral schema of thirteen fields. Every field carries three things — **value**, **confidence**, **evidence** — and a field cannot be constructed as PRESENT without evidence pointing to a file, a line number and the raw line.

aaa_enabled	http_server_enabled	ntp_servers	telnet_enabled
banner_present	https_server_enabled	snmp_v3_only	weak_ciphers
idle_timeout_seconds	logging_enabled	ssh_version	
logging_hosts	min_password_length		

Access lists use a structured vendor-neutral representation. Vendor-specific syntax never reaches the compliance rule engine — enforced as a forbidden import edge, not a convention.

3 The six non-negotiable rules

Six architectural rules constrain the system permanently. Each is enforced by a test or a type contract, so violating one breaks the build rather than merely breaking a promise.

RULE	HOW IT IS ENFORCED
1 • AI never issues a verdict	75 forbidden import edges. <code>comply→learn</code> , <code>comply→train</code> , <code>normalise→learn</code> and <code>report→learn</code> carry the weight. A <code>Suggestion</code> object has no <code>value</code> field to promote. The audit database enforces <code>CHECK (actor_type <> 'model' OR action = 'ai_suggested')</code> below Python.
2 • Evidence is mandatory	Enforced at <code>Field</code> construction — a PRESENT field without evidence cannot be built. Comment prefixes and literal blocks never become parse nodes, so a commented-out directive cannot produce a PRESENT field.
3 • Low confidence abstains	Confidence populations are floored separately and never compared. Deterministic and admin-confirmed matches are exactly 1.0 or nothing; <code>platform_default</code> has its own floor and is always marked INFERRED; only calibrated similarity is compared against the threshold. <code>UNCALIBRATED_SIMILARITY</code> forces UNKNOWN whatever the score.
4 • Remediation is never AI-generated	<code>remediate→learn</code> and <code>remediate→train</code> are forbidden. Commands are read from <code>snippets/</code> and never synthesised. A snippet requires <code>vetted_by</code> and <code>reference</code> , and a content-policy test refuses a vetter whose name looks automated.
5 • Rules and packs are data	<code>packs/</code> and <code>rules/</code> are YAML. A new pack version is written at runtime and activated without a restart — <code>clear_pack_cache()</code> makes the next parse use it in the same process.
6 • Offline-first	No device library may be imported. No network client in <code>ingest</code> , <code>parse</code> , <code>learn</code> , <code>train</code> or <code>prioritise</code> . Secrets are scrubbed before any text reaches an embedding model. An airgap setting makes the model loader fail closed rather than fetch.

3.1 The architecture is enforced, not documented

Fifteen architecture test files assert these properties from different angles: no ML library outside `learn`, no network capability, no vendor literal in a vendor-neutral layer, no raw configuration line in the training queue, and no path from the evaluation harness into any pipeline package. The forbidden edges are distributed across packages:

```
analyse 8 · audit 3 · comply 9 · ingest 6 · learn 8 · normalise 6  
parse 6 · prioritise 10 · remediate 8 · report 7 · train 4
```

The shape of the argument is always the same: **a layer that decides must not be able to see the layer that suggests, and a layer that suggests must not be able to reach the layer that decides.**

4 Innovation I — vendor packs as data

4.1 What a vendor pack is

A vendor pack is a YAML file describing how one platform's syntax maps to the canonical model. It holds detection signatures, identity patterns, comment prefixes, literal-block rules, and the patterns that read security fields.

```
vendor: acme-os  
version: 1  
patterns:  
- field: ssh_version  
  match: {type: regex, pattern: "^set ssh proto-version (\d+)"}  
  capture: {value: "$1", cast: int}  
  source: admin-trained  
  examples: ["set ssh proto-version 2"]
```

4.2 Why this was added

The problem statement asks for a *vendor-agnostic* auditor. There are two ways to build one, and the choice determines whether the system can ever be adopted.

APPROACH	CONSEQUENCE
Vendor logic in code an <code>if vendor == "cisco"</code> chain	Adding a platform means a code change, a review, a release and a redeployment. In a defence or enterprise setting that is a change-control cycle measured in weeks. The tool is permanently behind the network it audits.
Vendor logic as data NIRIKSHAK's choice	Adding a platform means adding a YAML file. No application code changes. An operator with no access to the source can extend coverage, and the extension is reviewable by anyone who can read a regular expression.

4.3 How it improves the system

- **Coverage grows without engineering.** Three platforms ship today — Cisco IOS, Arista EOS, Juniper Junos. A fourth requires no developer.
- **Packs are versioned and checksummed.** Every pack states a SHA-256 over its own bytes, verified on every load. A pack modified on disk is refused rather than silently trusted.
- **Every finding is reproducible.** A finding records the engine version, the rulepack version and the vendor pack versions that read its lines. A verdict is reproducible only if the data that produced it is identified.

- **Rollback is real.** Every pack version stays on disk. An activation record names which version is in force, so a deployment can return to an earlier parse behaviour exactly — because no pack file was ever modified in place.
- **Two roots, kept distinct.** `packs/builtin/` is repository content a reviewer read; `packs/trained/` is deployment state this installation learned. Committing the second would blur the one distinction the two roots exist to keep.

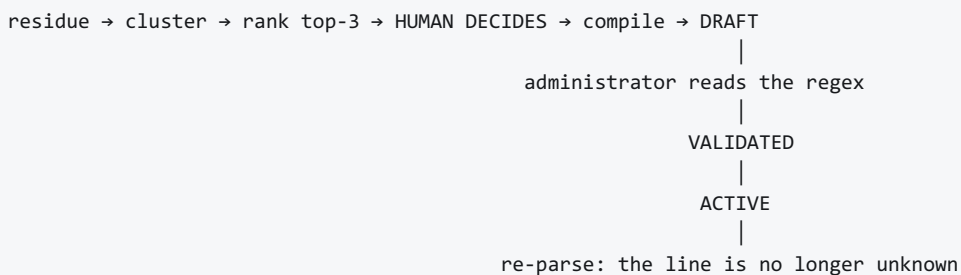
DESIGN CONSTRAINT: PATTERNS MUST BE BORING

Generated patterns are deliberately unambitious: tokenise the confirmed line, replace the captured token with `(\S+)`, escape the rest, anchor both ends. The compiler does not infer alternation, detect optional tokens, or merge two confirmations into one general pattern. *A pattern an administrator cannot read is one they cannot verify* — and this pattern will read every future configuration of that platform, for everyone.

5 Innovation II — the administrator confirmation loop

This is the project's central differentiator: the mechanism by which the system learns syntax it has never seen, without ever guessing.

5.1 The loop



STEP	WHAT HAPPENS, AND WHY
Residue	Every configuration node no pattern matched is recorded. This is the honest output of parsing: not an error, and not silently dropped.
Cluster	Unknown lines are grouped by token shape. One shape across thirty devices is one decision worth thirty — the loop scales by fleet size, not by line count.
Rank	Up to three candidate canonical fields are retrieved from the labelled-example index. These are <i>rankings</i> , explicitly never probabilities.
Confirm	An administrator confirms, corrects, or marks the line not security relevant. <code>confirmed_by</code> is taken from the authenticated identity and the field is rejected if sent in the request body — a caller able to set it could attribute their confirmation to a colleague.
Compile	The confirmed line becomes a pattern. The administrator says which token carries the value; the compiler never guesses whether the <code>2</code> in <code>ip ssh version 2</code> is a value or part of a command name.
Review	The generated expression is shown and is editable. This step is a separate HTTP call and cannot be skipped. An edited expression is re-validated, never trusted as typed; <code>.*</code> is refused by name.
Activate	The pack is validated against its own examples — a pattern that does not match the line it came from cannot go live — then written, checksummed and activated.

5.2 Verified: the system learns immediately

A learning claim is worthless if it needs a restart. This was tested end to end rather than asserted: a configuration is uploaded and audited, a mapping is added, and the *same bytes* are re-audited.

MEASURED

Unread lines drop from N to $N-1$ in the same process, with no restart and no redeployment. Three further properties are asserted in the same suite: a mapping can be created for a line the residue queue never held; an edit composes as withdraw-then-recompile with the full version lineage left on disk; and a hand-edited expression is still re-validated.

5.3 Where trust originates

The model is never the authority. Administrator confirmation is what creates the trusted mapping, and the system records that fact permanently: every admin-trained pattern retains the training example it was compiled from and the audit sequence number of the moment it was confirmed. Any mapping in any pack can be traced to the person who confirmed it and the moment they did.

5.4 Withdrawal is not deletion

An administrator can retire a mapping they no longer stand behind. The operation drafts a *new* pack version without the pattern, validates it and activates it. Three things are preserved:

- The version holding the pattern stays on disk — an audit already run under it remains explainable.
- The training example is untouched — a decision an administrator made is an event that happened, and unlearning a mapping is not the same act as pretending nobody ever confirmed it.
- Both halves are appended to the audit chain, carrying identifiers and a reason but never the configuration line.

Only admin-trained patterns may be withdrawn. A built-in pattern is repository content, removed by a reviewed commit, and the refusal lives in the backend rather than only in the interface.

6 Innovation III — abstention as a first-class result

Most audit tools report pass or fail. NIRIKSHAK reports **PASS**, **FAIL**, **UNKNOWN** and **NOT_APPLICABLE**, and treats the third as a result rather than as a missing answer.

6.1 Absence-aware evaluation

A missing directive is not automatically a failure. When a control is absent the engine asks three questions in order:

1. Does the platform support this control at all? If not — **NOT_APPLICABLE**.
2. Is there a documented default for this platform? If so, the field is asserted from that default and is **permanently marked INFERRED**, never presented as observed.
3. Is the absence even determinable? If support or default behaviour is unknown — **UNKNOWN**. The system abstains.

6.2 Why this matters more than it sounds

THE FAILURE THIS PREVENTS

A tool that cannot tell "this control is absent, therefore it fails" from "I could not determine this" will eventually report a device as compliant because it failed to parse the relevant line. That is a vulnerability formally signed off. Abstention is the mechanism that makes the difference visible.

6.3 Abstention is protected at the presentation layer too

The interface never lets UNKNOWN look like a weaker FAIL. UNKNOWN is rendered dashed and neutral slate — deliberately *not* amber — because abstention sits off the severity axis rather than at the bottom of it. FAIL is heaviest: a solid fill with reversed text. PASS is lightest, because a compliant control needs no attention.

The reasoning is operational: if the interface makes abstention look like a weaker failure, operators learn to filter it out, and Rule 3 is defeated at the presentation layer having survived everywhere else.

Every state pairs its colour with a text label and a distinct weight, because reports print in greyscale and a meaningful share of engineers have colour vision deficiency. Severity uses ink weight, never a second colour scale.

7 Innovation IV — the vetted remediation library

7.1 The principle

NIRIKSHAK **never generates commands**. There is no template, no format string that builds a command, no fallback to a similar platform, and no model in that path. Commands are *resolved* from a library of files on disk, keyed by `(vendor, os_family, rule_id)`, or there is no command.

7.2 What "vetted" enforces

Two fields are mandatory in both the JSON schema and the type contract:

- `vetted_by` — the **person** who checked the commands. An architecture test greps for names that look automated and fails the build if one appears.
- `reference` — the vendor document they checked it against, with a locator.

A snippet therefore cannot exist without a human who read a document and confirmed the commands against it. The operator sees who stands behind every command and what they read.

7.3 What a snippet carries

A command alone is not enough to paste into production. Each snippet carries the commands, a **rollback** (mandatory whenever the change is service-affecting), preconditions, verification commands, and an impact assessment naming service impact, reload requirement and lockout risk.

7.4 Lockout-aware ordering

Snippets are ordered by dependency first, then by lockout risk *ascending*, then by an order hint. A high-risk change is applied **last**. The reason is concrete: disabling Telnet before SSH is verified working is exactly how an operator is stranded outside the device they are fixing.

1. NRK-TIMEOUT-001 lockout=none hint=300
2. NRK-SSH-001 lockout=low hint=100
3. NRK-TELNET-001 lockout=high hint=900 ← depends on NRK-SSH-001

Real ordering produced over the shipped Cisco IOS snippets.

7.5 Current coverage

PLATFORM	RULES COVERED	NOTES
Cisco IOS	7 of 7	Complete
Juniper Junos	7 of 7	Telnet removal uses <code>commit confirmed 5</code> — the device rolls back automatically if the operator is stranded
Arista EOS	6 of 7	SSH protocol-version deliberately absent

THE DELIBERATE GAP

There is no Arista EOS snippet for NRK-SSH-001. That rule requires SSH protocol version 2, and EOS exposes no protocol-version setting to configure — there was no command to vet. The rule resolves to *"No vetted remediation is available for this platform and rule."* Writing a command there to complete the coverage table is precisely the failure this library exists to prevent.

7.6 Placeholders are deliberate

A command needing a site-specific value — a syslog collector, a time source, banner wording — carries it as `<SYSLOG-COLLECTOR-IP>`, never a plausible default, and the preconditions say so. NIRIKSHAK does not know a site's collector and must not invent one: a command that silently shipped a fleet's logs to an address nobody chose would be worse than no command at all.

7.7 Remediation is never applied

The system recommends; a human operator applies. There is no automatic remediation path, and none is planned.

8 Innovation V — the tamper-evident audit chain

8.1 Two databases, deliberately separate

operational store	audit store
<code>config_file, config_line</code>	<code>audit_log</code>
<code>line_cache, device</code>	<code>audit_chain_head</code>
<code>ingestion, app_user</code>	<code>schema_migrations</code>
<code>audit_run, finding</code>	
<code>finding_evidence</code>	
<code>unknown_line, training_example</code>	

The operational store holds configuration-derived content. **The audit store holds attestations about it, and nothing else.** The chain records *that* an audit ran, over which device, with which rulepack, and how many findings of each verdict — never a value, never a raw line. Keeping them in separate files makes that claim checkable by opening one.

8.2 How the chain works

`audit_log` is append-only, enforced by database triggers rather than by convention. Each record binds the hash of its payload to the hash of the record before it, so a retroactive edit anywhere breaks verification everywhere after.

`audit_chain_head` is a singleton that detects deletion of the chain's tail, which the links alone cannot.

Recorded actions: file ingested, file rejected, audit run, AI suggested, admin confirmed, admin corrected, pack created, pack activated, pack rolled back, report generated.

STATED LIMITATION

The chain is tamper-evident, not tamper-proof. It detects record modification, deletion, reordering, broken links and accidental corruption. It does *not* detect an attacker with unrestricted database write access who recomputes the complete unkeyed chain. This limitation has its own decision record rather than being left for someone to discover.

8.3 Evidence is stored as pointers, not copies

A finding's evidence is `(file_id, line_number)`, resolved through the line store to the exact stored text. A report therefore quotes the operator's own file rather than a transcription that could drift from it.

9 Where AI is used, why, and how it is contained

AI is used in exactly one place, for exactly one purpose, and it cannot reach a verdict from there.

9.1 The one place

COMPONENT	WHAT IT DOES
<code>api/learn/cluster</code>	Groups unrecognised lines by token shape. Deterministic — no model involved.
<code>api/learn/embedding</code>	Adapter over <code>sentence-transformers</code> with the <code>all-MiniLM-L6-v2</code> model, CPU-only. Turns a scrubbed configuration line into a vector.
<code>api/learn/index</code>	The labelled-example index: seed examples from the shipped packs, plus every mapping an administrator has confirmed.
<code>api/learn/suggest</code>	Retrieves up to three candidate canonical fields for an unknown line, ranked by similarity.
<code>api/learn/calibration</code>	The machinery to turn similarity into probability. Built, and deliberately not fitted — see below.

9.2 Why AI is used here and nowhere else

The task is "*this line looks like lines that meant X*". That is a similarity question over short technical strings, which embeddings answer well and rules answer badly — you cannot write a regular expression for "syntax I have never seen."

Every other task in the system is a task where a rule is *better*, not merely adequate: deciding a verdict, reading a known directive, resolving a command, ranking by exposure. Using a model for any of those would trade a verifiable answer for an unverifiable one.

9.3 How it is contained — five independent barriers

#	BARRIER	EFFECT
1	Import graph	<code>comply</code> , <code>normalise</code> , <code>remediate</code> and <code>report</code> may not import <code>learn</code> or <code>train</code> . The verdict layer cannot see the suggestion layer.
2	Type contract	A <code>Suggestion</code> carries a proposed <i>meaning</i> for a line. It has no value field — there is nothing on it to promote into a canonical field.
3	Confidence method	Every suggestion leaves the package marked <code>UNCALIBRATED_SIMILARITY</code> , which the contract treats as forcing the field to UNKNOWN <i>regardless of the score attached</i> .
4	Human gate	The only route from suggestion to verdict is: a human confirms it → it becomes pack data → the deterministic parser matches it next run.
5	Database constraint	<code>CHECK (actor_type <> 'model' OR action = 'ai_suggested')</code> — below Python, a model cannot be recorded as the actor for any action except making a suggestion.

9.4 Similarity scores are not probabilities

A DELIBERATE REFUSAL

The calibration machinery is built and no calibrator is fitted, because fitting one requires line-level ground truth that does not exist. A raw cosine similarity of 0.87 is *not* an 87% chance of being correct, and a frontend given a bare float will eventually render it as a percentage. So the API returns `rank`, the raw score, and an explicit `is_probability: false`; the interface labels these as rankings; and every suggestion abstains until a human confirms it.

9.5 Offline and privacy-preserving

- **Local model, CPU only.** `all-MiniLM-L6-v2` runs on the operator's machine. Target hardware is a standard 8 GB laptop with no GPU.
- **No cloud AI dependency.** Configuration data never needs to leave the operator network.
- **Secrets scrubbed before inference.** A line is redacted at the normalisation stage precisely because it will reach an embedding model, and nothing downstream un-scrubs it.
- **Fails closed.** With airgap enabled the loader must not reach the network even if weights are absent — the failure is a clean refusal rather than a hanging fetch.
- **Degrades honestly.** Without the model the training queue still works; it simply arrives unranked, and says which state it is in rather than presenting an empty suggestion list as "nothing similar found".

9.6 What AI is explicitly not used for

No compliance decisions inside AI prompts. No model-generated production CLI. No configuration chatbot. No model fine-tuning. No unnecessary cloud AI dependency. These are prohibitions in the project's permanent constraints, enforced by dependency choice and lint rule rather than by policy.

10 The technology stack, and why each choice

10.1 The stack

LAYER	CHOICE	ROLE
Language	Python 3.11	Backend, pinned <code>>=3.11, <3.12</code>
API	FastAPI + Uvicorn	28 HTTP endpoints
Contracts	Pydantic v2	Typed models that validate at construction
Storage	SQLite (two databases)	Operational store and audit store
Parsing	Own block parser + TextFSM, lxml, jsonpath-ng	Hierarchical configuration → <code>ConfigTree</code>
Data formats	YAML (PyYAML) + JSON Schema	Rules, packs, snippets
AI	sentence-transformers, FAISS, PyTorch (CPU)	The advisory branch only
Reporting	Jinja2 + WeasyPrint	HTML always; PDF where GTK is present
Frontend	React 18 + TypeScript + Vite + Tailwind	Operator interface
Security	scrypt, HTTP Basic, cryptography	Password hashing, auth, encryption at rest
Testing	pytest, Vitest, Testing Library, Ruff	1,887 + 34 tests

10.2 Why these, and not the alternatives

Python 3.11 — not Go, Rust or Java

The parsing, rules and ML ecosystems the problem needs are strongest in Python. `sentence-transformers`, `FAISS`, `TextFSM` and `ntc-templates` are all first-class here and would each need reimplementations elsewhere. 3.11 specifically for `tomllib`, exception groups and modern typing syntax; the version is pinned because a version drift discovered during a demo is an expensive way to learn about it.

FastAPI — not Django or Flask

Pydantic validation is built into the request layer, so a malformed request is rejected by the type contract rather than by hand-written checks. Django brings an ORM, migrations, templates and an admin site — none of which this project wants, since the storage layer is deliberately hand-written SQL against two separate databases. Flask would need the validation layer bolted on.

SQLite — not PostgreSQL or MySQL

This is the choice that most directly serves the requirements:

- **Airgap deployment.** A single file, no server process, no network listener. A server database is another daemon to harden in a secure facility.
- **Two databases are two files.** The claim "the audit store holds no configuration content" is checkable by opening one file. With one server holding both schemas, that separation is a convention rather than a fact.
- **Triggers enforce append-only** below the application layer.

- **The workload fits.** Single-operator, read-heavy, one audit at a time. Nothing here needs concurrent write scaling.

Own block parser — not ciscoconfparse, and not TextFSM alone

This is the most consequential deviation from the specified stack, and it has its own decision record. TextFSM and `ntc-templates` target the output of `show` commands: flat, tabular, record-oriented text. Running configurations are not that shape — they are hierarchical.

WHY THE BLOCK MODEL IS NOT OPTIONAL

The line `exec-timeout 10 0` means nothing without knowing which block encloses it. Under `line vty 0 4` it is a management idle timeout; under `line con 0` it is a different control entirely. TextFSM has no block model. Since deterministic parsing is the foundation the whole system rests on, a gap here propagates into *every* verdict.

The parser handles four syntax modes — indented blocks (Cisco, Arista), brace nesting (Junos), flat `set` paths (Junos, PAN-OS) and XML (PAN-OS) — and preserves parent/child context and exact source evidence, which is what Rule 2 requires. `ciscoconfparse` is not introduced unless a concrete requirement appears that this parser cannot support. TextFSM is retained where it fits.

YAML — not a database or a DSL

Rules, packs and snippets are configuration a human reads and reviews. YAML is diffable, reviewable in a pull request, and readable by an operator who is not a programmer. A database would make them invisible to code review; a custom DSL would need a parser, an editor mode and documentation before it earned anything.

all-MiniLM-L6-v2 — not a large language model

CONSIDERATION	WHY THE SMALL MODEL WINS HERE
Task shape	The task is similarity between short technical strings. A 22M-parameter sentence embedder is the right tool; an LLM is a generative model being used as a comparator.
Hardware	Runs on CPU on an 8 GB laptop — the stated target. An LLM needs a GPU or a server.
Determinism	The same line embeds identically between runs, so every metric computed over the index is reproducible. LLM output is not.
Failure mode	A weak embedding produces a poor ranking a human then corrects. A hallucinating LLM produces a confident wrong mapping — and the wrong-confident rate is this project's critical safety metric.
Auditability	A vector similarity is inspectable. A prompt response is not.

FAISS — not a vector database

An in-process library with no server, consistent with the airgap requirement. The index is small — seed examples plus confirmed mappings — and Pinecone, Weaviate or Qdrant would each add a network dependency to a system whose defining constraint is not having one.

WeasyPrint — not a headless browser

Renders the same Jinja2 HTML the interface serves, so the PDF and the web report cannot diverge. A headless Chromium would add a large binary and a process launch to a tool intended for restricted environments. WeasyPrint needs a GTK runtime, which is *probed live* rather than assumed: where it is missing the PDF endpoint answers 503 naming the missing libraries and never substitutes the HTML document with a different content type.

React + TypeScript + Tailwind + Vite

TypeScript matters more than usual here: the interface consumes a large typed API, and a wrong type declaration is invisible at runtime until something dereferences it and unmounts the application. Tailwind keeps the design tokens in one config rather than scattered across stylesheets, which is what makes a strict palette discipline enforceable. Vite gives fast builds and a dev proxy that keeps the API same-origin, so no CORS configuration and no API host baked into the bundle. Both typefaces are self-hosted with their OFL licences, because a font request at page load would break an airgapped deployment *and* disclose that the tool is in use.

10.3 What was deliberately excluded

No device-connection library (Netmiko, NAPALM, Paramiko, Scrapli, ncclient, pysnmp) — banned by lint rule and by an architecture test that scans both the source tree and the resolved environment. No ORM, no migration framework, no task queue, no message broker, no cloud AI client.

11 The interface, screen by screen

ui/ is a **pure consumer**. It never evaluates a rule, computes a verdict, scores exposure, ranks a finding or compares a baseline. Every number it shows was returned by the API or is a count of rows the API returned.

This section walks every entry in the sidebar: what happens inside it, and why it was built that way. The reasoning matters more than the layout — nearly every decision below exists to stop one specific failure, and the failure is named each time.

11.1 What is in the sidebar

The navigation is a drawer, closed until it is asked for. An operator working through one device should see that device, not a permanent list of the six places they are not looking. Inside, the entries are split into two groups, and the split is a claim about the product rather than a way to shorten a list.

#	ENTRY	STAGE	QUESTION IT ANSWERS
PIPELINE — THE BASE IDEA, IN EXECUTION ORDER			
1	Devices	Ingest · Parse	What configurations do we hold, and what was read from each?
2	Compliance	Evaluate	What failed or abstained across the fleet?
3	Remediation	Resolve	What do I type, and in what order?
4	Reports	Report	What document can I hand to someone?
BEYOND THE PIPELINE — CAPABILITIES BUILT ON TOP OF IT			
5	Vendor packs	How each platform is read	What does the system know, and what did we teach it?
6	Activity log	Every recorded action	What happened, when, and who did it?
7	Status	Installed capabilities	What can this deployment actually do?

WHY THE TWO GROUPS ARE SEPARATE

Pipeline is the problem statement, in execution order, and it gains nothing else. A screen that is not one of those five stages does not belong in that list however useful it is — otherwise the list stops meaning "*the product*" and starts meaning "*things we built*".

Beyond the pipeline is what is built on top of that base: how the platform descriptions are maintained, what has been recorded, and what this installation can actually do. Capabilities in their own right, not steps in a run.

Two screens sit outside the sidebar. The **landing page** has no navigation bar at all — one claim, one sentence, one way in; an operator who arrives there either signs in or has no business further along. The **sign-in screen** is reached from it. The drawer's footer carries the signed-in identity, the role, and a sign-out control beside it, because the drawer is where someone goes when they are finished and closing it again to leave is a small indignity at the end of every session.

11.2 The three levels of zoom

Fleet (which devices need attention) → device (what is wrong with this one) → finding (why do you claim that, and what do I type). Each level answers one question and links down for the rest, and the levels are reached *without*

leaving the screen: a row expands in place rather than navigating. The finding detail was built before any dashboard, because it is the atom of the product and everything else is composition.

11.3 Devices

INGEST · PARSE

What happens

Two panes. The **left** is the fleet: an upload control, a search box, and every configuration held. Each row carries the hostname, the platform, and a one-line audit summary — *3 fail · 0 unknown · Sep 04, 09:43*, or *Never audited*. The **right** is one device: a *Run audit* button and five tabs.

Selecting a device swaps the right pane and nothing else. The URL follows the selection so refresh and the back button still work, but no navigation redraws the page.

Why two panes

An operator working through a fleet is holding a position in a list. Navigating to a device page and back loses it, and by the fifth device they have lost track of which ones they have already done. Two panes cost one screen and save that.

The five tabs — the pipeline again, at device scale

TAB	WHAT HAPPENS, AND WHY
Overview	Identity as read from the file — hostname, vendor, OS family, OS version, model, configuration file id — then the latest run's verdict counts, the rulepack version that produced them, and the previous runs. <i>Why</i>: a field that was not read says why — "no identity pattern matched", "not read from the configuration" — rather than showing a blank. A blank is ambiguous between "the device does not have this" and "we failed to read it", and those need different responses.
Findings	Every rule evaluated, filterable by verdict (All · Fail · Unknown · Pass · N/A). A row expands in place to show the expected value, the observed state, the observed value, the confidence <i>and the method that produced it</i>, the evidence viewer with the cited line in its surrounding context, and the remediation panel. <i>Why</i>: this is the atom of the product. The confidence <i>method</i> is shown beside the number because "1.0, deterministic" and "0.72, similarity" are not the same kind of number and must never be read on one scale.
Needs review	The confirmation loop: unrecognised lines with their ranked candidate meanings. Confirm, correct, or mark not security relevant → compile → read the generated expression → activate. <i>Why here</i>: the loop lives inside the device it concerns because that is where the operator has the context to judge a line. <code>logging host 10.0.0.1</code> means something different when the rest of the file is visible.
Remediation	The fix plan for this device. Steps with a vetted command start expanded, showing commands, rollback, preconditions, verification and impact together. <i>Why expanded</i>: §10 requires "never the command alone". The panel previously collapsed everything behind a row reading "Vetted command available", which told an operator the thing they came for exists and then made them ask for it.
Report	The document, behind a gate — see §11.6.

Three details inside Needs review

A NON-ADMIN GETS A REFUSAL, NEVER AN EMPTY QUEUE

The screen states it plainly: *"The confirmation loop writes into a vendor pack, so the backend restricts it to administrators. Your account can read this device's findings and evidence, but not decide what an unrecognised line means."* An empty list would be a claim about the data. This is a claim about their role, which is the truth.

A MISSING MODEL IS ANNOUNCED, NOT SILENTLY ABSORBED

When the embedding model is unavailable the screen says *"No suggestions are being produced"* and names the reason. Zero suggestions and "nothing was searched" look identical on screen and mean opposite things. The queue still works unranked — and it tells the operator that is what they are looking at.

ACTIVATION SAYS WHAT IT DID NOT DO

The confirmation toast reads *"Re-run the audit to apply it to this device."* The pack is live immediately and the next audit will use it, but *this* device's stored findings were computed under the old pack. A bare "done" would imply the screen in front of the operator had updated. It has not.

11.4 Compliance EVALUATE

What happens

The most recent evaluation of every device, one row each, with its verdict counts and the rulepack version that produced them. Expanding a row shows that run's findings that are **not PASS** — verdict chip, control, severity and the cited line. Above the table sits a peer-baseline strip: a plain-language summary and an outlier count.

Why only the non-PASS findings

This screen answers *"what needs attention."* A compliant control needs none. Listing passes would triple the row count and bury the three findings that matter under the twenty that do not.

Why the baseline strip prints a sentence instead of a zero

THE MOST DANGEROUS SHAPE OF BUG IN THIS PRODUCT

Every cohort in the current corpus is below the minimum size of five, so no baseline is established. The strip prints that in words rather than showing "0 outliers" — because **an empty outliers list rendered as "no drift" would tell an operator the fleet is uniform when the truth is that nothing was comparable.** A refusal that looks like a clean result is worse than an error, because nobody investigates it.

Why the interface never computes the baseline itself

The arithmetic lives entirely in the backend. A second implementation in TypeScript would be a second answer nobody reconciled, and the two would drift — at which point the screen and the report would disagree about the same fleet.

11.5 Remediation RESOLVE

What happens

Every device with at least one failing check. Expanding a row shows that device's plan: how many checks are failing, how many have a vetted command, and the ordered steps.

Why a device with no vetted command still appears

Omitting it would understate the work by exactly the amount nobody has vetted yet. If a rule has no snippet for a platform, that is a real gap in remediation coverage — hiding the row would make the fleet look more fixable than it is.

Why the ordering is computed rather than filtered

Steps are sequenced by dependency first, then by lockout risk *ascending*. The Telnet step comes **last**, after SSH is verified working, because disabling the management path before its replacement works is how an operator is stranded outside the device they are fixing.

\$10 forbids offloading ranking onto filters the operator must drive themselves. A sortable column would put the responsibility for not locking themselves out on the person least able to see the dependency graph.

11.6 Reports REPORT

What happens

One row per audited device, showing when it was evaluated and against which rulepack. Expanding gives either the rendered document inline or a locked panel naming precisely what is outstanding. A PDF download sits beside it.

The gate opens when three things are true

1. The device has been audited.
2. Every unrecognised line has been decided — *including* "not security relevant".
3. Every vetted command has been marked reviewed.

Why gate at all

A report asserts that a configuration was audited, that the lines nobody recognised were put to a person, and that the commands an operator will paste into a device were looked at. Until those are true, the document claims more than the work behind it.

Why the panel says it is not a server refusal

The backend renders a report for any persisted run. The gate is NIRIKSHAK's own workflow rule, and conflating "we will not yet" with "the server cannot" would send somebody debugging the wrong thing entirely.

A WEAKNESS STATED ON SCREEN RATHER THAN HIDDEN

The review marks are `localStorage` notes in the operator's own browser. They never reach the hash-chained log, because no remediation-approval endpoint exists. The screen says exactly that, rather than letting an operator believe their approval was recorded somewhere durable.

The PDF path

Where the native renderer is missing, the endpoint answers a clear 503 naming the missing libraries — never the HTML document served with a PDF content type. A file that claims to be a PDF and is not is a failure that surfaces somewhere far from its cause.

11.7 Vendor packs BEYOND THE PIPELINE

What happens

One box per platform, each showing the active version and two counts weighted differently — human-verified in ink, because that is what changes on this screen, and built-in receding. Opening a box gives three sections.

SECTION	WHAT IS INSIDE
Human-verified first, because it changes	Every mapping an administrator confirmed: the expression in full, the line it was compiled from, who confirmed it and the audit sequence number of when . Add, replace and withdraw.
Built-in	Mappings shipped with the repository, authored against a real corpus file and reviewed like any other code. Read-only.
Version history	Every version of this pack on disk — version, parent, status, origin, pattern count and checksum.

Why the split is the whole point of the screen

The two kinds carry different warrants. A built-in pattern was reviewed as code against a corpus. A human-verified one was confirmed by a named person at a known moment. **Merging them into one list would destroy the only signal that tells an operator how far to trust a row.**

Why built-in is read-only, and why the backend refuses it too

Repository content changes by reviewed commit, not by a button. The refusal lives in the backend rather than only in the interface, because a client that decided for itself which patterns were removable would be one release away from removing a shipped one.

Why adding a mapping takes two steps

First the operator gives the line and **clicks which token carries the value**. The compiler never guesses whether the `2` in `ip ssh version 2` is the value or part of the command name — only a person knows that, and a form that inferred it would be making that judgement silently.

Then the generated expression is shown and is *editable* before activation. An edited expression goes back through the compiler and is re-validated, never written into a pack as typed; `.*` is refused by name. This pattern will read every future configuration of that platform, for everyone, until somebody withdraws it — so a pattern nobody read is a pattern nobody can vouch for.

Why "withdraw" and not "delete"

Withdrawing drafts a *new* pack version without the pattern, validates it and activates it. Three things survive:

- The version that held the pattern stays on disk, so an audit already run under it remains explainable.
- The training example is untouched — a decision an administrator made is an event that happened, and **unlearning a mapping is not the same act as pretending nobody ever confirmed it.**
- Both halves are appended to the audit chain, carrying identifiers and a reason but never the configuration line.

Why the whole screen is admin-only

A confirmation changes how every future device of that platform is parsed, for everyone, permanently. A non-admin is told that plainly rather than shown an empty list — an empty list would read as a claim about the data rather than about their permissions.

11.8 Activity log BEYOND THE PIPELINE

What happens

The hash-chained audit trail, read-only and filterable by action across ten types: file ingested, file rejected, audit run, AI suggested, admin confirmed, admin corrected, pack created, pack activated, pack rolled back, report generated.

Each row shows the sequence number, the timestamp, the action, **who** performed it with their actor type, and the subject. A verification badge sits at the top: *CHAIN VERIFIED* with the record count, or *CHAIN FAILED* in solid FAIL weight.

Why six actions are weighted heavier than the rest

Admin confirmed, admin corrected, pack created, pack activated, pack rolled back and file rejected are the actions where **a person changed what the system will believe next**. Those are the records an auditor reads first, so they carry the ink and the rest recede.

Why the actor type is printed on every row

It makes Rule 1 visible and checkable. A model can only ever appear against `ai_suggested` — enforced by a database `CHECK` constraint below Python, not by convention. An auditor can read the log and confirm for themselves that no model ever decided anything.

Why there is no endpoint to add a record

Records are appended by the services that perform the actions. An endpoint that could inject one would attest to whatever a client claimed rather than to what the system did — which would make the chain a log of assertions rather than of events.

11.9 Status BEYOND THE PIPELINE

What happens

CARD	WHAT IT REPORTS
Ranked suggestions	Embedding model availability, split into <i>package installed</i> and <i>weights present</i> , with the model name and a plain-language summary.
PDF export	Whether the renderer can serve, and exactly which native libraries are missing if not.
Vetted remediation	How many snippets loaded and the library's version digest.
Abstention floors	All three confidence thresholds, each labelled with the population it governs.
Service	Status, version, airgap setting, phase, and each database schema version.

Why probed live rather than cached

The stack can be installed while the service is running. A cached negative would keep reporting the absence of something that is now present, until somebody restarted the process for reasons they could not have guessed.

Why the model state is two flags, not one

"Package installed, weights absent" and "package absent" fail differently and are fixed differently — one is a `pip install`, the other a download. A single combined "unavailable" would send an operator to the wrong fix.

Why three thresholds appear where one would look tidier

Rule 3 has two floors governing different populations. Deterministic matches are exactly 1.0 or nothing. Platform defaults have their own floor *and* their own assigned value. Only calibrated similarity is compared against the main threshold. A readout naming one number would suggest a single threshold governs everything — which is precisely the misreading that the separation exists to prevent.

Why the library count is on this screen at all

So an operator can tell "no remediation is available for this rule" from "the library failed to load". Those produce the same empty panel and need opposite responses.

WHAT IS DELIBERATELY ABSENT

There is no "parser: online" tile. The backend exposes no such probe, and **a green light nobody measured is worse than no light** — it manufactures confidence out of nothing, which is the one thing this product must never do.

11.10 Principles that hold across every screen

- **Evidence is always one interaction away.** Any finding traces to its source line without leaving context, showing surrounding lines with the matched span marked. Fields asserted from a platform default carry a visible INFERRED marker that cannot be suppressed, so an operator always knows the difference between observed and inferred.
- **An empty result and a refusal never look alike.** Five distinct states — loading, empty, blocked, error, abstained — because a capability that silently returns nothing is indistinguishable from a clean result, and that confusion is how a fleet gets signed off unaudited.
- **Nothing navigates away.** Rows expand in place, so an operator working through a list never loses their position in it.
- **Colour never carries meaning alone.** Every state pairs its colour with a text label and a distinct weight, because reports print in greyscale and a meaningful share of engineers have colour vision deficiency. FAIL is heaviest and draws the eye first; PASS is lightest; UNKNOWN is dashed neutral slate — deliberately *not* amber, because abstention sits off the severity axis rather than at the bottom of it.
- **Density is correct; clutter is not.** One separation mechanism per table, tabular numerals so columns align, and no chart that a sentence or a sorted table would carry better. The review screen is the deliberate exception: spacious, one line at a time, because a cramped review produces careless confirmations and a careless confirmation enters a vendor pack permanently.
- **Role checks here are UX, not security.** The backend enforces every rule independently: admin endpoints answer 403, and a resource belonging to another user answers 404 rather than 403, so an unauthorised caller learns nothing about which identifiers exist.

12 Security posture

PROPERTY	IMPLEMENTATION
No live device access	No connection library may be imported. Enforced by lint rule and by an architecture test scanning source and environment.
Offline operation	Configuration data never needs to leave the operator network. No network client in the pipeline packages.
Secret scrubbing	Credential-adjacent strings are redacted before any text reaches an embedding model.
Password storage	scrypt.
Authorisation	Admin endpoints answer 403 to a non-admin. A resource belonging to another user answers 404 rather than 403 , so an unauthorised caller learns nothing about which identifiers exist.

Credential handling	HTTP Basic; the browser holds the token in <code>sessionStorage</code> , scoped to the tab, never <code>localStorage</code> . The limitation is stated in the code rather than hidden.
Interface role checks	UX controls, not security. The backend enforces every rule independently.
Audit trail	Hash-chained, append-only by database trigger, covering AI suggestions, administrator corrections, pack changes and audit results.
Pack integrity	SHA-256 over each pack's own bytes, verified on every load.

13 Measured results

READ THIS FIRST

Every configuration in the corpus is synthetic — written by the team to be realistic, not captured from a real network. A synthetic file contains exactly the shapes its author thought to include, so the parser is scored against its author's imagination rather than against the field. These numbers are real measurements of a synthetic sample. They are **not** real-world accuracy and may not be presented as such.

13.1 Vendor detection

4 correct of 4, 0 wrong, 0 abstained — measured against the vendor the manifest records, which the author noted when writing the file and which no parser produced.

13.2 Field extraction, by vendor

Vendors with a parsing pack and detection-only vendors are never pooled. A combined recall figure would be dominated by packs nobody has written.

VENDOR	PACK	N	OK	WRONG	MISS	ABSTAINED	PRECISION	RECALL
cisco	yes	26	11	0	4	11	100.0% / 11	73.3% / 15
arista	no	13	0	0	3	10	n/a	0.0% / 3
juniper	no	13	0	0	4	9	n/a	0.0% / 4

Arista and Juniper have detection but no parsing patterns — an honest state, not a placeholder. The platform is recognised and its fields abstain.

13.3 The critical safety metric

0 WRONG-CONFIDENT ANSWERS	11 TOTAL ASSERTIONS	100% EVIDENCE INTEGRITY
-------------------------------------	-------------------------------	-----------------------------------

HOW TO READ THE ZERO

A zero here is **not** evidence of accuracy. The system asserts a value only where a deterministic pattern matched, and it currently holds patterns for one platform. A system that asserts little cannot assert much wrongly. This figure must be read alongside the miss counts above, not instead of them.

13.4 Compliance verdicts, Cisco (the platform with a pack)

EXPECTED \ ACTUAL	PASS	FAIL	UNKNOWN	N/A
pass	8	0	0	0
fail	0	3	3	0
unknown	0	0	0	0

PASS: precision 100.0% / 8, recall 100.0% / 8. FAIL: precision 100.0% / 3, recall 50.0% / 6. The three FAILs that became UNKNOWN are recall losses where the system abstained rather than asserting — which is the intended failure direction.

13.5 Evidence integrity

Scored separately from value accuracy, because a correct value carrying a citation that does not support it is a failure rather than a rounding error. Cisco: 11 checked, 11 correct, 0 pointing at the wrong line, 0 missing.

13.6 Ground-truth provenance — stated, not hidden

NOT INDEPENDENT GROUND TRUTH

0 of 4 labels have been reviewed by a second person, and the Cisco labels were written by the author of the Cisco parsing patterns. Correlated error between parser and ground truth is therefore *not visible* in the Cisco numbers: a field misunderstood while writing the pattern would be misunderstood the same way while writing the label. Arista and Juniper carry no such conflict, because no parsing pattern has ever been written for either.

14 What is built — feature inventory

CAPABILITY	STATE
Configuration ingestion	Upload, validation, content hashing, blob storage, rejection recording. Binary, malformed, oversized and empty files are refused and the refusal is itself recorded.
Vendor detection	Weighted signature matching from pack data. Adding a platform's detection is a data change.
Hierarchical parsing	Four syntax modes. Produces a <code>ConfigTree</code> plus residue.
Device identity	Hostname, model, OS version, serial extracted from pack identity patterns.
Normalisation	13 canonical fields, each with value, confidence and evidence. Absence-aware.
Compliance engine	7 canonical rules, deterministic, four verdict classes with evidence.
ACL analysis	Interval logic detecting shadowed, redundant and overly permissive entries. Built and tested; the corpus contains no ACL to run it on.
Exposure prioritisation	Built. Abstains without ACL and interface data, and says so rather than offering a severity-sorted list in its place.
Peer baselines	Built. Every cohort is below the minimum size of five, so no baseline is established and no device is called an outlier.
Similarity layer	Clustering, embedding, index, top-3 retrieval. Ships uncalibrated by decision.
Confirmation loop	Queue → confirm → compile → DRAFT → review → activate. Two steps, never one.
Pack lifecycle	Versioning, checksums, activation record, rollback, withdrawal. Live in-process, no restart.
Remediation	20 vetted snippets across 3 platforms, dependency and lockout-risk ordering, rollback and impact on every entry.
Reporting	Self-contained HTML with cited lines and its own disclosures. PDF where GTK is present.
Audit chain	Append-only, hash-linked, verifiable, with a head singleton.

Evaluation harness	Precision, recall, correct-abstention, wrong-confident, evidence integrity, per vendor and per field. Exits non-zero only when a measurement cannot be made honestly — never because a number is low.
Interface	9 screens, drawer navigation, device workspace, vendor pack management, evidence viewer, activity log, capability status.
Authentication	HTTP Basic, script hashing, admin/user roles, per-resource ownership.

15 What is deliberately not claimed

This is the most important section in the report. Every item below is a capability that is **built and tested** but has no data to run on. The machinery is real; the output is an honest refusal.

NOT CLAIMED	WHY
Coverage against CIS, NIST SP 800-53, DISA STIG or ISO/IEC 27001	Every rule ships an empty framework list. Writing a control identifier without having read the benchmark would be inventing it.
Absence-aware evaluation accuracy	No platform default and no capability claim ships, so that branch has never fired on real data.
ACL detection rates	The corpus contains no access list in any split. The analyser has never seen a parsed one.
Exposure scores or a priority ranking	Exposure needs interfaces and access lists; the corpus has zero of both.
Peer-baseline outliers	Every cohort is below the minimum size of five.
Held-out generalisation	The metric is defined over the held-out vendor's commands; reading them needs an XML parser awaiting a sample independent of the holdout.
Top-3 mapping accuracy or calibrated confidence	No line-level ground truth exists and no calibrator is fitted. Every similarity score is a ranking, never a probability.
Real-world accuracy	Every corpus file is hand-written by one author.
Independent ground truth	The labels are unreviewed, and the Cisco labels share an author with the Cisco parsing patterns.

All nine trace to eight entries in the sourcing backlog: ACL-bearing configurations · vendor capability and default documentation · XML samples that do not compromise the holdout · framework control-ID sources · broader vendor diversity · vendor remediation documentation · line-level ground truth · a corpus written by more than one author.

None can be closed by writing code, and none may be closed by inventing data.

15.1 The sealed holdout

One vendor — **PAN-OS** — is held out entirely for the generalisation experiment. Two files are recorded in the corpus manifest with `split: holdout`. **They have never been opened** — not read, not hashed, not parsed, at any point in any phase.

The seal is structural rather than procedural:

- The evaluation harness raises before a file handle is opened for a held-out split.
- PAN-OS has no active vendor pack, and XML tree building raises rather than returning an empty tree — so a held-out file cannot enter the pipeline even by accident.
- Architecture tests scan the learn, train and prioritise packages for the path fragments `holdout/` , `corpus/holdout` , `panos` and `paloalto` , with docstrings stripped first: explaining the rule is expected, constructing a path is not.

The reason is single-use: once those files have been studied to build a parser, top-3 accuracy on them measures memory rather than generalisation. The experiment can be run once, and it has not been spent.

16 Known defects and open work

Fifteen numbered defects have been found and recorded. Thirteen are fixed; **two remain open** and are documented rather than hidden.

ID	DESCRIPTION AND REASONING	STATUS
DEF-3	<code>device_id</code> is the configuration file's content hash, so it identifies <i>this configuration</i> rather than the physical device across time. Fixing it means redefining an identifier that every finding, every audit row, every report and the evaluation already carry — changing it would move a measurement. The real consequence is recorded rather than hidden: <i>a configuration re-uploaded after an edit counts as a second device</i> in its cohort. Nothing anywhere presents a content hash as a stable device identity — the report names its field <code>config_file_id</code> and the interface labels devices by hostname.	OPEN
DEF-8	<code>NRK-TIMEOUT-001</code> passes <code>exec-timeout 0 0</code> — a session that never expires is reported as compliant. The correct check is "at most 600 seconds <i>and</i> greater than zero", and the check specification examines one field with one operator from a closed set; <code>lte</code> cannot express it. Fixing it needs a compliance-engine contract change belonging to a rules phase with its own decision record. No corpus device uses <code>exec-timeout 0 0</code> , so no current measurement depends on it either way.	OPEN

Representative fixed defects, to show the class of problem being caught:

ID	DESCRIPTION	STATUS
DEF-4	A configuration flag could have turned every abstention into a pass	FIXED
DEF-9	Arista pack did not declare <code>!</code> as a comment prefix; 23 of 57 residue lines were comments	FIXED
DEF-10	Field provenance hard-coded BUILTIN, so a learned mapping would claim to be vendor-shipped	FIXED
DEF-11	Pack versions ordered by string comparison; <code>1.0.10</code> sorted below <code>1.0.9</code>	FIXED
DEF-13	Pack checksums were declared and never verified against file bytes	FIXED
DEF-14	The audit endpoint never appended an AUDIT_RUN record to the chain	FIXED
DEF-15	Detected device identity never reached the canonical model in the live pipeline	FIXED

17 Engineering discipline

17.1 Decision records

23 architecture decision records capture 60 numbered decisions, each stating the context, the decision and its consequences. Where a decision deviates from the specified stack — the block parser, for instance — the deviation is argued rather than made silently.

17.2 Testing

LAYER	WHAT IT GUARDS
Architecture (15 files)	75 forbidden import edges, no device libraries, no ML outside <code>learn</code> , no vendor literals in neutral layers, the sealed holdout, content policy.
Unit (1,034 tests)	Parsers, rules, resolvers, the pattern compiler, the chain.
Integration	The full pipeline over the real API, including the learning loop end to end.
Golden	Fixed inputs to fixed expected outputs.
Frontend (34 tests)	Honesty (no invented data reaches the screen), contracts (response shapes), interaction, routing, pack management.

17.3 The content policy

A document may cite an identifier and a locator, never transcribed vendor prose. The policy is enforced by tests over the rules, packs and snippets trees rather than left to reviewer memory.

17.4 The honesty tests

A distinctive category: frontend tests that assert the interface does *not* display things. That no framework identifier appears while every rule ships an empty list. That an undetermined exposure is stated as undetermined rather than shown as a rank. That a refused capability reads as a refusal rather than as an empty queue. These test the absence of invention, which is the property the whole system is built around.

18 Summary

NIRIKSHAK is a vendor-agnostic network security compliance auditor built on one commitment: **a deterministic rule engine issues every verdict, and machine learning is confined to suggesting meanings for syntax nobody has taught the system yet** — a suggestion that reaches a verdict only after a human confirms it and it becomes data.

Five things distinguish it:

1. **Vendor packs as data.** Coverage grows by adding a YAML file, not by a code release. Versioned, checksummed, and reversible.
2. **The confirmation loop.** Unknown syntax is not guessed at; it is put to an administrator, and their answer is compiled into a pack that takes effect in the same process, with no restart. Verified end to end.
3. **Abstention as a result.** UNKNOWN is a first-class verdict, protected from PASS by type contracts and from looking like FAIL by the visual design.

4. **Vetted remediation.** Commands are resolved from a library where every entry names the person who checked it and the document they read — never generated — and are ordered so a lockout-risking change comes last.
5. **An audit trail that is honest about its own limits.** Hash-chained and append-only, and documented as tamper-evident rather than tamper-proof.

The system is measured rather than claimed. On the synthetic evaluation split it records 100% precision on the platform it parses, 0 wrong-confident answers of 11 assertions, and 100% evidence integrity — and the report states plainly that a zero wrong-confident rate on 11 assertions is not evidence of accuracy, that the corpus is synthetic, and that the ground truth is unreviewed.

Nine capabilities are built and produce honest refusals because the data to exercise them has not been sourced. Two defects are open and documented. One vendor is sealed and has never been read.

*Prefer a small auditable claim over a large unverified one.
An honest UNKNOWN is a result; a guessed PASS is a liability.*